

초안 — 검토용

VIGENT 팀 협업 기획안 (통합본)

버전 v0.9

생성일 2026-07-27

대외비 — 외부 배포 금지

목차

신호등 범례 — ● [완료] 실측 완료 · ● [진행중] 부분·진행 · ● [미착수] 미착수·미검증 (색이 안 보이면 대괄호 텍스트로 판단)

1. 00. 시작하기 전에 (3분 요약)
2. 01. 현황 — 숫자로 보기
3. 10. 비전(L1) 트랙 기획안 ★ 핵심 문서 1
4. 20. 에이전트(L2) 트랙 기획안 ★ 핵심 문서 2
5. 30. 통합 로드맵
6. 40. Claude Code 사용법 ★ 바이브코딩 규칙
7. 50. 완전초보 온보딩 (가장 자세하게)
8. 60. 협업 규칙 — Git
9. 70. 역할 분담과 주간 루틴
10. 80. 용어 사전

※ 정확한 페이지 번호는 PDF 뷰어의 페이지 표시를 참고하세요. (브라우저 인쇄 시 하단 페이지번호 옵션 사용 가능)

00. 시작하기 전에 (3분 요약)

이 문서는 팀원 전원(대표 포함)이 가장 먼저 읽는 문서입니다. 바이브코딩·팀 프로젝트가 처음이어도 괜찮습니다.

VIGENT가 뭘 만드는 회사인가 (5줄)

- VIGENT는 공장·건설현장의 CCTV 영상을 AI가 보고 위험을 잡아내는 산업안전 플랫폼입니다.
- 카메라가 "위험구역에 사람이 들어왔다 / 보호구를 안 썼다 / 불·연기가 났다"를 자동으로 감지해 경보합니다. (이것을 **비전(Vision, L1)**이라 부릅니다.)
- 감지 결과로 위험성평가서 같은 안전 서류를 AI가 자동 작성합니다. (이것을 **에이전트(Agent, L2)**라 부릅니다.)
- 즉 "보는 눈(비전) + 판단하는 머리(에이전트)"가 한 시스템으로 붙어 있습니다.
- VIGENT는 보조 감시 도구입니다 — 법으로 정해진 안전장치(방호장치 등)를 대체하지 않고, **추가로 지켜보는 신호**를 줍니다.

지금 어디까지 왔나 (3줄)

- 비전·에이전트 코드와 자체 테스트(82개 통과)는 갖춰졌고, 동시부하 크래시(F-14) 같은 큰 안정성 문제는 해결했습니다.
- 하지만 **실제 현장 CCTV로 정확도를 잴 적이 거의 없습니다**(실영상 검증 1건). 공개 데이터셋 수치만 있습니다.
- 감지 클래스는 3클래스 가동(person·PPE·fire/smoke) + forklift(지게차)는 재학습 대기입니다.

앞으로 뭘 할 건가 (3줄)

- 비전(L1)**: 현장 영상으로 정확도(오탐·미탐)를 재고, 부족한 클래스(안전모 미착용 등)를 재학습해 KISA 인증 기준($F1 \geq 0.90$)을 향합니다.
- 에이전트(L2)**: AI가 만든 안전서류의 품질을 계속하고, 잘못된 법령 인용(환각)을 막고, 비전 감지와 자동 연동합니다.
- 두 트랙이 만나 **파일럿(시범 설치) → 유상 납품**으로 갑니다. 상세 단계는 10·20 문서에 있습니다.

문서 읽는 순서

순서	문서	누가 꼭 읽나
1	00_시작하기_전에 (이 문서)	전원
2	01_현황_숫자로_보기	전원 (특히 대표)
3	10_비전트랙_기획안	비전 담당·대표
3	20_에이전트트랙_기획안	에이전트 담당·대표
4	30_통합_로드맵	전원
5	40_클로드코드_사용법	전원 (개발 참여자 필수)
6	50_완전초보_온보딩	신규 팀원 필수 (설치 처음이면)
7	60_협업규칙_git	코드 만지는 전원
8	70_역할분담과_주간루틴	전원
9	80_용어사전	모르는 단어 나올 때마다

- 개발이 처음이면: 00 → 01 → 50(온보딩) → 40 → 60 순서를 권장합니다.
- 대표·사업 담당이면: 00 → 01 → 10 → 20 → 30 을 보면 전체 그림과 일정이 잡힙니다.

⚠ 보안 주의 (반드시)

- 이 리포(코드·수치·문서)의 **내용을 외부에 공유하지 마세요**. 대외비입니다.
- 공개 시연·외부 발표 전에는 반드시 **특허 출원**을 먼저 합니다. (특히 RIG 모듈은 특허 대상이라 세부 동작은 팀 내에서도 [대외비] 로 다룹니다.)
- `.env` (비밀키)·모델 가중치·현장 영상은 절대 **git에 올리지 않습니다** (상세: 60_협업규칙_git.md).

개발 도구는 Claude Code 하나로 통일합니다. 다른 IDE·플러그인은 쓰지 않습니다 (이유·사용법: 40_클로드코드_사용법.md).

01. 현황 — 숫자로 보기

Phase 1 실행 감사 결과입니다. 모든 숫자에 출처 파일과 확인 명령어가 붙어 있습니다. 근거 없는 값은 [측정 필요] 입니다. 표기: ● 실행·완료 / ● 부분·진행중 / ● 미착수·미검증

신호등 요약 (모듈별 한 줄)

모듈	상태	한 줄
비전 검출(3클래스)	●	person·PPE·fire/smoke 가동, 공개셋 수치만 있음(현장 미검증)
비전 forklift(지게차)	●	재학습 대기(오탁으로 비활성)
비전 추적·안정성	●	ByteTrack 교체·F-14 크래시 해소·90분 소크 합격
비전 낙상	●	광역 CCTV 해상도로 자동 낙상 불가(근접 카메라 필요)
에이전트 문서생성	●	위험성평가서 실행작, 나머지 생성기 부분/스텝
에이전트 품질계측	●	골든 채점·회귀 엔진·홀드아웃 15케이스(합격 11/15)
L1→L2 자동연동	●	빈도 통계 재사용 수준(항목 자동 O/X 아님) — 20 문서 S3 참조
RIG 모듈	●	상태기계-테스트 있음 / 실영상 1건 / 하드경보 배선 불가
협업 인프라	●	427커밋·게이트·설치문서 있음 / .env·weights는 각자 준비 필요

표 1-A. 비전(L1) 현황

항목	현재 수치	출처 경로	확인 명령어	상태
감지 클래스	3클래스 가동(person-PPE-fire/smoke) + forklift 재학습 대기	docs/COMMERCIALIZATION_READINESS.md §1	sed -n '22,26p' docs/COMMERCIALIZATION_READINESS.md	●
PPE 착용	Hardhat 76·Mask 75·Vest 67 (recall only, precision 미측정 · MPS/공개셋)	동 §1:24	sed -n '24p' docs/COMMERCIALIZATION_READINESS.md	● 공 공 공
PPE 미착용(NO-Hardhat)	R56.1 (미착용 ≈44% 놓침 · recall only, precision 미측정)	동 §1:24	동	● 오 중
fire-smoke	공개셋 D-Fire mAP@50 80.13%	동 §1:25	sed -n '25p' ...	● 공 공 공
person mAP	[측정 필요] (readiness "공개셋 mAP 미재확인") · 웹캠 recall 100%(recall only)	동 §1:22	클래스별 벤치 재실행 필요	●
forklift	비활성(F-7, 정탐 conf 0.002) · PoC box mAP 8.83→58.62%	동 §1:26	sed -n '26p' ...	●
zone_intrusion(타일)	소형작업자 recall 0→89~95%(실영상, recall only · CPU +415ms/f)	동 §1:23	sed -n '23p' ...	●
클래스별 F1 / KISA 캡	[측정 필요] — P/R/F1 미산출(V-02 티켓)	—	티켓 V-02 스크립트로 산출	●
추론 지연	타일 +415ms/f(CPU) · 클래스별=[측정 필요]	동 §1:50	grep 415ms docs/COMMERCIALIZATION_READINESS.md	●
측정 환경	현재 Apple MPS(CUDA 아님) → 배포 환경(CUDA/엣지)에서 재측정 필요	동 §1:50	동	●
가중치	18개·812MB(git 미포함)	vigent-core/weights/	du -sh vigent-core/weights ; ls vigent-core/weights	●
포즈·추적	포즈= ergonomics.py (yolov8n-pose) · 추적=ByteTrack	vigent-core/rfdetr_service.py	grep -n ByteTrack vigent-core/rfdetr_service.py	●
파이프라인 진입점	worker.py (loop)→ rfdetr_service.py (detect)	vigent-core/worker.py	grep -n '_loop' vigent-core/worker.py	●
AGPL 잔존	배포 트리 강카피레프트 0건 ✓	benchmarks/results/licenses.md	grep -i '0건' benchmarks/results/licenses.md	●
재연결·워치독	RTSP 지수백오프 재연결 · deploy/watchdog.sh · F-14 해소(fd606c1)	vigent-core/worker.py :41 · deploy/watchdog.sh	grep -n RECONNECT vigent-core/worker.py	●
연속구동	90분 합성 소크 합격 / 실카메라 24h [측정 필요]	audit/soakmon_2026-07-20_validate90m.md	sed -n '1,15p' audit/soakmon_2026-07-20_validate90m.md	●
미해결	현장 오탐율 미측정 · 낙상 불가(B8) · 엣지 RK3588 포팅 코드 없음	docs/P3_BACKLOG.md B8	grep -n '26~64px' docs/P3_BACKLOG.md	●

이게 무슨 뜻이냐면: 검출 수치는 공개 데이터셋 기준이고 대부분 recall만 있습니다(오탐을 뜻하는 precision은 안 했습니다). 지금 값은 맥(MPS)에서 난 것이라, 실제 배포 장비(CUDA GPU/엣지)에선 다시 측정해야 합니다.

표 1-B. 에이전트(L2) 현황

항목	현재 수치	출처 경로	확인 명령어
Scribe(평가서)	819줄, 실동작	vigent-core/agents/scribe.py	wc -l vigent-core/agents/scribe.py
Guard(검출지휘)	391줄	agents/guard.py	wc -l ...
Copilot(법령)	121줄, 화이트리스트 게이트	agents/copilot.py	wc -l ...
Dispatcher	149줄, fail-closed	agents/dispatcher.py	wc -l ...
Analyst	164줄	agents/analyst.py	wc -l ...
Coach	12줄 = 스텝	agents/coach.py	wc -l ...
골든셋(체크리스트)	3건(crane-confined-press)	eval/golden/*_checklist.json	ls eval/golden/*_checklist.json
골든셋(T1홀드아웃)	15케이스 · 합격 11/15 (≥80점 AND 치명적0)	datasets/goldens/T1/ · 커밋 d93ec6c	git log --oneline\ grep /15
최근 채점 (crane)	커버 3/3·위험수준 3/3·위계 3/3·법령 4/6·활각 4	eval/golden/regression_baseline.json	python3 scripts/golden_regression.py
법령 화이트리스트	규칙 21개 · legal_whitelist.py 189줄	config/corpus/safety_citations.json	python3 -c "import json;print(len(json.load(open('config/corpus/safety_citations.))
환각 차단	audit_citations (화이트리스트 밖 조문 차단, F-11) / 차단 로그 잔존=[측정 필요]	agents/scribe.py :285 · legal_whitelist.py	grep -n audit_citations vigent-core/agents/scribe.py
LLM 의존	외부 API(OpenAI/Anthropic) · 온프렘(ollama) 제거됨 · 단 법령·위계·빈도는 규칙기반 (LLM 무관)	vigent-core/llm_provider.py :1-13	sed -n '1,13p' vigent-core/llm_provider.py
테스트	15파일 · 82개 전부 통과(실행함)	tests/	python3 -m unittest discover -s tests
L1→L2 연동	빈도 통계 재사용 (count→_likelihood), 요청 시 events 전달 — 항목 자동 O/X 아님	agents/scribe.py :585,129 · data_engine.py :6	sed -n '129,130p;585,586p' vigent-core/agents/scribe.py

이게 무슨 뜻이냐면: 에이전트 6개 중 4개는 실동작, Coach는 스텝(12줄)입니다. AI 판단의 품질을 재는 채점 체계는 갖춰졌지만(홀드아웃 11/15), 비전 감지가 체크리스트에 자동으로 O/X를 채우지는 않습니다(빈도만 재사용) — 이 연결이 20 문서 S3의 핵심 과제입니다.

표 1-C. RIG 모듈 현황

항목	현재 수치	출처 경로	확인 명령어	상태
상태기계	rig_monitor.py · rig_replay.py (세부 전이·임계 = [대외비])	vigent-core/rig_monitor.py	ls -la vigent-core/rig_*.py	● 로직
단위 테스트	monitor 5 + replay 6 = 11개(82개에 포함)	tests/test_rig_*.py	grep -c 'def test_' tests/test_rig_*.py	●
실영상 검증	낙상경보 타이밍 1건(t=38→ALARM t=38.21s)	docs/COMMERCIALIZATION_READINESS.md :30	sed -n '30p' ...	● n=1
라이브 배선	하드 fall-alarm 배선 불가 확정(B8)	docs/P3_BACKLOG.md B8	grep '배선 불가' docs/COMMERCIALIZATION_READINESS.md	●
HW(LED·부자)·음성	설계 단계(구현 코드 미확인) = [측정 필요]	—	grep -rI 'buzzer\ LED' vigent-core/	●

이게 무슨 뜻이냐면: RIG는 핵심 로직·테스트는 있으나 실영상 검증이 1건뿐이고, 현재 카메라로는 자동 하드경보를 못 겁니다. LED·부자·음성은 아직 설계입니다. (특히 대상이라 세부는 대외비)

표 1-D. 리포·협업 인프라 현황

항목	현재 수치	출처 경로	확인 명령어	상태
총 커밋	427	git	git log --oneline \ wc -l	●
브랜치	main(clean) + 6개	git	git branch -a	●
미커밋 변경	0건	git	git status --short	●
README·CLAUDE·gitignore	모두 존재	루트	ls README.md CLAUDE.md .gitignore	●
requirements	완전 편(17페이지 ==)	requirements.txt	grep -c '==' requirements.txt	●
신규 막힘 ① .env	없음 → .env.example 복사·키 입력 필요	.env.example	ls -la .env .env.example	●
신규 막힘 ② weights	812MB git 미포함 → 수동 복사(없으면 COCO 풀백, F-8)	.gitignore	git check-ignore vident-core/weights	●
신규 막힘 ③ 설치문서	있음 docs/ONBOARDING.md	docs/ONBOARDING.md	ls docs/ONBOARDING.md	●
신규 막힘 ④ 진입점	run.sh / uvicorn main:app (cwd=vident-core)	run.sh ·ONBOARDING \$2.5	grep -n uvicorn docs/ONBOARDING.md	●

이게 무슨 뜻이냐면: 코드·이력·문서는 잘 관리돼 있고, 신규 팀원이 실제로 막히는 건 `.env` 와 `weights/` 두 개뿐입니다(둘 다 git으로 안 와서 각자 준비).

지금 당장 팔 수 있는가? (정직한 답 3줄)

- 아니요. 현장 실영상으로 정확도(오탐·미탐)를 쟀 적이 없고(1건뿐), KISA 인증 기준(F1≥0.90)도 아직 측정 전입니다.
- 가장 가까운 판매 가능 형태는 "위험구역 침입 경보 단일 기능 파일럿(시범)"입니다 — 이걸 자체 검증이 가장 앞서 있습니다(타일 recall 89~95%).
- 먼저 필요한 것: 현장 카메라 1대 + 실영상 오탐율 측정 + GPU 장비. (상세: docs/PILOT_PROPOSAL.md · docs/INFRA_REQUIREMENTS.md)

10. 비전(L1) 트랙 기획안 ★ 핵심 문서 1

(1) 이 트랙이 파는 것

- 현장 CCTV로 위험을 자동 감시합니다 — 위험구역 침입·보호구 미착용·화재/연기를 사람이 놓쳐도 24시간 잡아냅니다.
- 중거 프레임을 남깁니다 — 아차사고 분석·안전교육 자료로 씁니다.
- 감지 클래스: 3클래스 가동(person·PPE·fire/smoke) + forklift(지게차) 재학습 대기.

(2) 상용화 단계 정의 (S0~S6)

단계	이름	완료 판정 기준(숫자)	현재 상태	남은 작업	예상 소요(추정)
S0	모델 성능 확보	4클래스 전부 오프라인 F1 ≥ 목표치	3클래스 공개셋 recall만 있음(F1 미산출), forklift 비활성	클래스별 P/R/F1 산출(V-02), forklift 재학습	데이터 의존
S1 ▶	실환경 검증	실제 현장 CCTV에서 오탐률·미탐률 측정치 확보	실영상 1건뿐, 오탐률 미측정	현장 카메라 영상 확보 → 오탐/미탐 실측	현장 의존(1~2주+)
S2	무중단 안정성	24h 연속 무크래시 + 스트림 자동복구	90분 합성 소크 합격, F-14 해소, 재연결 구현 / 실카메라 24h 미실행	실카메라 24h 소크 (SOAK_24H_CHECKLIST)	GPU·현장 확보 후 1주
S3	패키징·폐쇄망 배포	인터넷 없는 PC에서 설치→구동 성공	미검증	Docker 이미지 + 오프라인 설치 절차 검증	[측정 필요]
S4	KISA 지능형 CCTV 인증	인증 기준 F1 ≥ 0.90 충족·신창·취득	F1 미측정	갭 산출(V-02)→개선→신창	장기(수개월)
S5	조달 진입	TTA GS 등급 + 조달 등록	미착수	S4 이후	장기
S6	유상 납품	첫 유상 계약	미착수	파일럿 성공 후	장기

▶ 현재 위치 = S0 마무리 ~ S1 진입 문턱 (코드·자체검증은 됐고, 현장 실측이 다음 관문).

각 단계 상세:

- S0** — 증거물: 공개셋 벤치 수치(docs/COMMERCIALIZATION_READINESS.md §1), 가중치 18개(vigent-core/weights/). 선행: 없음(완료 근접). 병렬: S2 안정성과 병렬 가능. 실패 리스크: F1을 안 재서 "성능 충분"이라 착각 → 대비책: V-02로 클래스별 F1을 먼저 숫자로 확정.
- S1** — 증거물: 현장 오탐/미탐 측정 리포트(아직 없음). 선행: S0의 가동 클래스. 병렬: 불가(실영상 없이는 시작 불가). 실패 리스크: 현장 카메라 각도·조명이 나빠 수치 폭락 → 대비책: 파일럿 초기 1~2주 튜닝 기간을 계약에 명시(docs/PIL0T_PROPOSAL.md).
- S2** — 증거물: audit/soakmon_2026-07-20_validate90m.md (90분 합격), tools/soak_monitor.py. 선행: S1과 병렬 가능. 실패 리스크: MPS 개발환경 특성으로 부하 시 위커 hang → 대비책: 배포 GPU(CUDA)에서 재측정(MPS 이슈는 CUDA서 미발생).
- S3** — 증거물: 없음. 선행: S0~S2. 실패 리스크: 폐쇄망에서 모델·의존성 누락 → 대비책: weights 동봉 오프라인 설치 스크립트 사전 검증.
- S4** — 증거물: 없음. 선행: S1~S3 필수. 실패 리스크: NO-Hardhat F1이 0.90에 크게 못 미침(현재 R56) → 대비책: V-03(재학습)을 가장 먼저 착수(리드타임 최장).
- S5/S6** — S4 취득 후. 리스크·대비책은 30_통합_로드맵.md 에서 통합 관리.

(3) 마일스톤 → 티켓 분해 (S0→S2)

★ 병목 3개(V-01·V-02·V-03)를 최상단 고정. 의존: V-01 → V-02 → V-03 → V-04 → V-06

[V-01][입문용] weights 공유 경로 확정 + 가중치 부재 시 fail-fast (F-8)

왜 하는가 : 신규 팀원·새 장비에서 가중치 없이 커먼 COCO로 조용히 풀백해 정확도가 급락함(과거 F-8 사고). 팀 확장의 1번 장애물.

담당 : (미정)

난이도 : ★ 예상 : 2~4시간

시작 조건 : weights 812MB 실물 접근 가능

완료 조건 : [] 공유 위치(클라우드/USB) + 복사 절차가 50 온도당 문서에 1개로 확정

[] 가중치 없을 때 서버가 조용히 풀백하지 않고 명확히 멈추거나 경고(현재 로그 표시 확인·강화)

건드리면 안 되는 파일 : vigent-core/weights/ 내부 파일(가중치 자체)

Claude Code 지시문 : "가중치 부재 시 동작을 조사해줘. rfdetr_service.guard가 weights 없을 때 어떻게 풀백하는지 코드로 확인하고, '조용한 COCO 풀백' 대신 기동 시 경고나 fail-fast로 바꾸는 최소 변경안을 먼저 계획으로 보여줘. F-8 관련 기존 코드(로그 표시)를 재활용하고, 게이트(테스트 82) 유지."

[V-02] 클래스별 P/R/F1 산출 스크립트 + KISA 갭 표

왜 하는가 : 지금은 recall만 있어 오탐(precision)과 F1을 모름 → KISA(F1≥0.90) 진척을 측정 불가.

담당 : (미정)

난이도 : ★★ 예상 : 4~6시간

시작 조건 : 평가용 라벨 데이터셋 경로 확보(data/)

완료 조건 : [] person·PPE·fire/smoke 각 P/R/F1 숫자 출력

[] KISA 0.90 대비 클래스별 갭 표 자동 생성

[] 측정 환경(장비·해상도) 리포트에 기록

건드리면 안 되는 파일 : vigent-core/rfdetr_service.py(추론 로직)

Claude Code 지시문 : "평가셋으로 person·PPE·fire/smoke 클래스별 precision·recall·F1을 산출하는 스크립트를 tools/에 만들어줘. 기존 benchmarks/ 벤치 코드를 재활용하고, 결과에 측정 환경(MPS/CPU·해상도)을 병기하고 '배포(CUDA)서 재측정 필요'를 리포트에 명시. 코드 수정 전 계획 먼저."

[V-03] NO-Hardhat 리콜 개선 계획 (데이터 수집 → 재학습)

왜 하는가 : 안전모 미착용을 44% 놓침(R56) = 안전상 가장 위험한 약점. 재학습은 리드타임 최장이라 가장 먼저 착수.

담당 : (미정)

난이도 : ★★★ 예상 : 계획 4~6시간(수집·학습은 별도 장기)

시작 조건 : 없음(계획 수립부터)

완료 조건 : [] 필요 데이터 종류·수량 정의(미착용 장면·각도·조명)

[] 재학습 파이프라인(train_safety.py) 실행 요건 정리(CUDA 필요)

[] 목표 F1과 예상 리드타임 명시

건드리면 안 되는 파일 : vigent-core/weights/(기존 가중치)

Claude Code 지시문 : "NO-Hardhat recall 56%를 올리기 위한 재학습 계획을 세워줘. 필요한 미착용 데이터의 종류·수량, tools/train_safety.py 실행 요건(학습은 MPS 불가·CUDA 필요), 목표 F1과 리드타임을 문서로 정리해줘. 코드는 수정하지 말고 계획만."

[V-04] 실카메라 24h 소크 실행 (S2)

왜 하는가 : 무인 24시간 무크래시를 실장비로 실증해야 S2 통과.

담당 : (미정) · 난이도 ★★ 예상 : 관측 1일 + 준비 2~4시간

시작 조건 : 실 RTSP 카메라 + GPU 장비, V-01 완료

완료 조건 : [] soak_monitor 리포트 크래시 0·메모리 누수 없음 [] 재연결 복구 100%

건드리면 안 되는 파일 : tools/soak_monitor.py(관측만, 수정 불필요)

Claude Code 지시문 : "docs/SOAK_24H-CHECKLIST.md 절차대로 실카메라 24h 소크를 준비·실행하는 명령을 단계별로 알려줘. 서버는 관측만(코드 수정 금지)."

[V-05] [입문용] 현장 오탐 샘플 수집·분류 (S1 준비)

왜 하는가 : 오탐율 측정의 원자료. 초보자도 영상 보고 오탐/정탐 라벨 가능.

담당 : (미정) · 난이도 ★ 예상 : 2~4시간

시작 조건 : 현장 영상 일부

완료 조건 : [] 오탐/정탐 분류 표 1개 [] 오탐 유형 상위 3개 정리

건드리면 안 되는 파일 : 없음(데이터 작업)

Claude Code 지시문 : "현장 영상에서 검출 결과를 프레임별로 뽑아 오탐/정탐을 사람이 분류할 수 있는 표를 만드는 스크립트를 tools/에 만들어줘. 계획 먼저."

[V-06] CUDA 배포 재측정 (S1/S2 정합)

왜 하는가 : 현재 수치는 MPS 기준 → 배포 CUDA에서 성능·지연 재확인 필요.

담당 : (미정) · 난이도 ★★ 예상 : 4~6시간

시작 조건 : NVIDIA GPU 장비, V-02 스크립트

완료 조건 : [] CUDA에서 클래스별 지연·F1 재측정 [] MPS 대비 표

건드리면 안 되는 파일 : vigent-core/device.py(CUDA 자동선택, 수정 불필요)

Claude Code 지시문 : "NVIDIA 장비에서 device.py가 CUDA를 자동 선택하는지 확인하고, V-02 스크립트로 클래스별 지연·F1을 재측정해 MPS 결과와 비교 표를 만들어 줘."

[V-07] forklift 재학습 데이터 요건 정의 (S0)

왜 하는가 : 협착(proximity) 기능이 forklift 비활성으로 사실상 무동작. 재학습이 전제.

담당 : (미정) · 난이도 ★★ 예상 : 3~5시간

시작 조건 : 없음 · 완료 조건 : [] 필요 데이터·수량 정의 [] PoC(8.83~58.62) 재현 요건

건드리면 안 되는 파일 : vigent-core/weights/forklift_*

Claude Code 지시문 : "forklift 재학습에 필요한 데이터 종류·수량과 기존 PoC 재현 요건을 문서로 정리해줘. 코드 수정 없이."

[V-08] 엡지(RK3588) 포팅 타당성 조사 (S3 준비)

왜 하는가 : 현장 저전력 배포 옵션 검토. 현재 포팅 코드 없음.

담당 : (미정) · 난이도 ★★★ 예상 : 조사 1일

시작 조건 : 없음 · 완료 조건 : [] 지원 모델·변환(rknn) 가능성 [] 성능 추정 [측정 필요] 표기

건드리면 안 되는 파일 : 없음(조사)

Claude Code 지시문 : "RK3588 엡지 배포 타당성을 조사해줘 - 현재 모델(RF-DETR)의 rknn 변환 가능성과 예상 제약을 정리하되, 측정 안 된 성능은 [측정 필요]로 표 기."

의존 관계: V-01 → V-02 → V-03(핵심 경로) · V-02 → V-06 · V-01 → V-04 · V-05 → S1 · V-07 · V-08 은 병렬. [입문용]: V-01, V-05.

(4) 이 트랙의 3대 리스크

1. **현장 정확도 폭락** — 공개셋은 좋아도 현장(조명·각도·소형작업자)에서 미탐/오탐 급증.
 - **조기 경보:** V-05 오탐 분류에서 오탐율이 예상보다 크게 높거나, 소형작업자 미탐이 반복되면.
2. **NO-Hardhat 재학습 지연** — 데이터 수집·라벨·학습(CUDA)이 길어 S4(KISA)가 밀림.
 - **조기 경보:** V-03 데이터 수집이 2주 이상 정체되면.
3. **배포 환경 불일치** — MPS(개발)와 CUDA/엡지(배포)의 성능·안정성 차이로 재작업.
 - **조기 경보:** V-06 CUDA 재측정에서 지연·정확도가 MPS와 크게 다르면.

20. 에이전트(L2) 트랙 기획안 ★ 핵심 문서 2

(1) 이 트랙이 파는 것

- AI가 안전 서류를 자동 작성합니다 — 위험성평가서를 감지 이벤트로부터 초안 생성(법령 근거 인용 포함).
- 잘못된 법령 인용(환각)을 차단합니다 — 화이트리스트 밖 조문은 막습니다.
- 안전관리자의 반복 문서 작업을 줄여 시간을 돌려줍니다.

(2) 상용화 단계 정의 (S0~S6)

단계	이름	완료 판정 기준(숫자)	현재 상태	남은 작업	예상 소요 (추정)
S0	파이프라인 구현	6에이전트 전부 실동작 + 산출물 1건 end-to-end	● 4개 실동작·Coach 스텝(12줄), 위험성평가서 1건 실동작	Coach 구현 or 범위 확정	2~3일
S1	품질 계속 체계	골든셋 N건 + 채점 엔진 + 평균 점수 기록	● 회귀 엔진 + 홀드아웃 15케이스(합격 11/15)	골든 확대·절대품질 감사 재검수	진행중
S2	환각 법령 통제	화이트리스트 밖 조문 인용 0건 + 차단 로그 남음	● 차단 로직 있음(F-11) / 차단 로그 잔존 [축정 필요]	차단 로그 확인·환각 4건 원인 분석	3~5시간
S3	L1 연동	비전 이벤트 → 체크리스트 자동 반영 경로 실동작	● 미완 — 현재 빈도 통계 재사용(count→_likelihood), 항목 자동 O/X 아님	라이브 감지→항목 O/X 자동반영 배선(A-01)	[축정 필요]
S4	현장 수용성 검증	실제 안전관리자 N명 리뷰 통과 + 수정요청 반영	● 미착수	관리자 리뷰 세션	현장 의존
S5	폐쇄망 대응	외부 API 없이 온프레임 LLM으로 동일 품질 재현	● 핵심(법령·위계·빈도)은 규칙기반=오픈라인 동작 / 텍스트 LLM은 외부 API(OpenAI·Anthropic)뿐, 온프레임(ollama) 제거됨	종합의견용 온프레임 LLM 재도입·로컬 VLM 대안	1~2주
S6	유상 납품	첫 유상 계약 / 구독 전환	● 미착수	파일럿 후	장기

▶ 현재 위치 = S0~S1 병행 (파이프라인 대부분 실동작 + 품질 계속 시작. 단 S3 연동은 미완).

각 단계를 건너뛰면 어떤 사고가 나는가

- S0 생략 → 산출물이 end-to-end로 안 나와 데모조차 불가.
- S1 생략 → 품질을 못 재서 코드 바꿀 때마다 **몰래 나빠져도 모름**(회귀).
- S2 생략 → 잘못된 법령 조문이 고객 문서에 그대로 실림(신뢰·법적 리스크).
- S3 생략 → 비전이 잡은 위험이 서류에 자동 반영 안 됨 → "AI 자동화"라는 핵심 가치 상실(수동 입력과 다를 바 없음).
- S4 생략 → 현장 안전관리자가 안 쓰는 서류를 만들게 됨(수용성 0).
- S5 생략 → 폐쇄망(인터넷 차단) 공장에서 종합의견 생성 불가.

S3-S5 판정 근거 (감사 실측)

- S3: agents/scribe.py:129,585 가 이벤트의 count (빈도)를 _likelihood 로 변환해 쓰고, 감지 규칙→적정성 X 매핑은 요청 시 events를 넘겨받아 처리.
data_engine.py:6 "빈도 집계로 재사용" 명시 → 라이브 자동 O/X 파이프라인 아님 = 미완. 확인: `sed -n '129,130p;585,586p' vigent-core/agents/scribe.py`
- S5: llm_provider.py:1-13 — provider=openai/anthropic(둘 다 외부), ollama(온프레임) 제거됨(2026-07-14). 단 법령·위계·빈도는 규칙기반이라 LLM 없이 동작. 확인: `sed -n '1,13p' vigent-core/llm_provider.py`

(3) 마일스톤 → 티켓 분해 (S0→S3)

★ A-01(S3 연동)을 A 트랙 최상단 고정 — 감사에서 L1→L2 자동연동이 미완으로 확정됨. 의존: A-01 → A-02 · A-03 → A-04

[A-01] 비전 이벤트 → 체크리스트 항목 자동 O/X 반영 (S3, 최우선)
왜 하는가 : 현재는 빈도만 재사용 → "AI 자동화" 핵심 가치가 비어 있음. 감지→서류 자동반영이 이 트랙의 심장.
담당 : (미정)
난이도 : ★★★ 예상 : 6시간(설계) + 배선 별도
시작 조건 : data_engine.log_event·scribe.build_checklist 흐름 이해
완료 조건 : [] 감지 이벤트가 해당 체크리스트 항목을 자동 X로 표시(빈도만이 아니라 항목 매핑)
[] 미감지 요건은 '수동확인'으로 유지(조용한 놓침 방지)
[] 회귀 러너(golden_regression) 통과 유지
건드리면 안 되는 파일 : eval/golden/regression_baseline.json(먼저 근거 없이 갱신 금지)
Claude Code 지시문 : "data_engine.log_event → scribe.build_checklist 경로를 조사해줘. 지금은 이벤트 빈도(count)만 재사용하는데, 감지된 규칙이 체크리스트의 해당 항목을 자동으로 X로 반영하도록 바꾸는 설계를 먼저 계획으로 보여줘. 미감지 항목은 '수동확인' 유지. golden_regression 회귀 없게. 코드 수정 전 계획 승인받기."
VIGENT 대외비 · 초안—검토용 v0.9

[A-02] 환각 차단 로그 실증 + 환각 4건 원인 분석 (S2)

왜 하는가 : 차단 로직은 있으나 로그가 실제 남는지 미확인. crane 골든서 환각 4건 관찰됨.

담당 : (미정) · 난이도 ★★ 예상 : 3~5시간

시작 조건 : legal_whitelist.audit_citations 위치

완료 조건 : [] 화이트리스트 밖 조문 인용 시 차단 로그가 실제 남는지 실증

[] crane 환각 4건(osha38 등)의 원인 분류

건드리면 안 되는 파일 : config/corpus/safety_citations.json(코퍼스)

Claude Code 지시문 : "legal_whitelist.audit_citations이 화이트리스트 밖 조문을 차단할 때 로그가 실제 남는지 확인하고, eval/golden crane 채점의 환각 4건 원인을 분석해줘. 코드 수정 전 계획 먼저."

[A-03][입문용] 골든셋 1건 추가 + 채점 (S1)

왜 하는가 : 골든 체크리스트 3건뿐 → 계속 표본 확대. 초보자가 기존 양식 따라 작성 가능.

담당 : (미정) · 난이도 ★ 예상 : 3~4시간

시작 조건 : eval/golden/TEMPLATE.md

완료 조건 : [] 신규 공정 1건 골든 작성 [] golden_regression에 반영·채점 [] 베이스라인은 근거와 함께만 갱신

건드리면 안 되는 파일 : 기존 item_01~03 골든

Claude Code 지시문 : "eval/golden/TEMPLATE.md 양식으로 새 공정 골든셋 1건을 작성하는 절차를 안내하고, scripts/golden_regression.py로 채점해줘. 베이스라인 갱신은 근거 명시 후에만."

[A-04][입문용] Coach 에이전트 범위 확정 (S0)

왜 하는가 : Coach가 12줄 스텝 → 구현할지/범위 축소할지 결정 필요.

담당 : (미정) · 난이도 ★ 예상 : 2~3시간

시작 조건 : agents/coach.py·base.py 읽기

완료 조건 : [] Coach가 할 일 정의 or '보류' 명시 [] S0 판정에 반영

건드리면 안 되는 파일 : 다른 agents/*.py

Claude Code 지시문 : "agents/coach.py(12줄 스텝)와 base.py를 읽고, Coach가 맡을 역할을 정의하거나 '현 단계 보류'로 판정하는 계획을 제시해줘. 코드 수정 전."

[A-05] 위험성평가서 절대품질 감사 재검수 (S1)

왜 하는가 : 채점 베이스라인은 '회귀 감지 기준선'이지 '정답 확정' 아님. 법령 4/6 등 절대 타당성 미검증.

담당 : (미정 / 안전 전문가 협업) · 난이도 ★★ 예상 : 리뷰 세션 1회

시작 조건 : crane/confined/press 산출물

완료 조건 : [] 감사가 촉발 점수 확정 [] 개선 필요 항목 목록화

건드리면 안 되는 파일 : 없음(검토)

Claude Code 지시문 : "crane/confined/press 골든 산출물을 감사 재검수용으로 정리하고, 촉발(위험요인·법령·위계) 점수를 채울 표를 만들어줘."

[A-06] TBM·아차사고 생성기 상태 정리 (S0)

왜 하는가 : 위험성평가서 외 생성기는 부분/백로그 → 실동작 범위 명확화.

담당 : (미정) · 난이도 ★★ 예상 : 3~4시간

완료 조건 : [] 각 생성기 실동작/미구현 구분 [] 라우트-기능 매핑

건드리면 안 되는 파일 : agents/scribe.py 핵심 로직

Claude Code 지시문 : "TBM·관리체계·아차사고 생성기의 실제 동작 여부를 라우트와 코드로 확인해 실동작/미구현 표를 만들어줘. 조사만."

[A-07] 온프레임 LLM 대안 조사 (S5)

왜 하는가 : 패쇄망 공장은 외부 API 불가. 종합의견용 온프레임 LLM 재도입 필요.

담당 : (미정) · 난이도 ★★★ 예상 : 조사 1일

완료 조건 : [] 온프레임 후보(로컬 LLM) + 품질/자원 비교 [] 규칙기반만으로 커버되는 범위 명시

건드리면 안 되는 파일 : llm_provider.py(조사 단계)

Claude Code 지시문 : "패쇄망용 온프레임 LLM 후보를 조사하고, 현재 규칙기반만으로 커버되는 문서 범위와 LLM이 꼭 필요한 부분(종합의견)을 구분해줘. 코드 수정 없이."

[A-08] 법령 코퍼스 커버리지 점검 (S2)

왜 하는가 : 코퍼스 규칙 21개 → 대상 공정 대비 커버리지 확인 필요.

담당 : (미정) · 난이도 ★★ 예상 : 3~4시간

완료 조건 : [] 21규칙이 커버하는 공정/미커버 공정 목록 [] 보강 우선순위

건드리면 안 되는 파일 : config/corpus/safety_citations.json(먼저 계획 없이 편집 금지)

Claude Code 지시문 : "config/corpus/safety_citations.json의 21개 규칙이 커버하는 공정과 빠진 공정을 정리해줘. 조사만."

의존 관계: A-01 → A-02 (연동 후 환각검증) · A-03 → A-05 (골든 확대 후 감사검수) · A-04 · A-06 · A-07 · A-08 병렬. [입문용]: A-03, A-04.

(4) 비전 트랙과의 연결점 (어느 A가 어느 V에 막혀 있나)

A 티켓	막혀 있는 V 티켓	이유
A-01(이벤트→체크리스트 자동반영)	V-01(가중치 fail-fast)	자동반영이 실패 있으면 검출이 풀백 없이 제대로 돌아야 함(F-8)
A-02(환각·차단로그)	—	독립(법령 로직은 검출과 무관)
A-05(강사 재검수)	V-05(현장 오탐 샘플)	실제 감지 결과라야 서류 품질이 현장 의미를 가짐
A-07(온프레임 LLM)	V-08(엣지 포팅)	폐쇄망·엣지 배포는 함께 묶여 감(온프레임 환경)

이 트랙의 3대 리스크

- 1. 환각(잘못된 법령 인용)이 고객 문서에 실림 — 신뢰·법적 리스크.
 - 초기 경보: A-02에서 차단 로그가 실제로 안 남거나, 환각 건수가 골든 확대 시 늘면.
- 2. L1↔L2 연동 미완이 방치됨 — "AI 자동화" 가치가 비어 파일럿 설득력 하락.
 - 초기 경보: A-01이 계속 뒤로 밀리고 데모가 여전히 수동 입력이면.
- 3. 폐쇄망 대응 공백 — 공장 다수가 인터넷 차단 → 외부 API 의존이 납품 장벽.
 - 초기 경보: 파일럿 후보가 "인터넷 안 됨"을 조건으로 내거는데 온프레임 대안이 없으면.

30. 통합 로드맵

비전(V)·에이전트(A) 두 트랙을 한 장으로 병합합니다. 날짜가 아니라 **선행 조건** 기준으로 배치합니다(현장·데이터·GPU 확보 시점에 좌우되므로).

한 장 타임라인 (주 단위 · ASCII 간트)

```

      주1   주2   주3   주4   주5~
V-01 가중치   #####
V-02 P/R/F1   #####
V-03 NO-Hardhat ##### (수집·재학습은 이후 장기) →
V-05 오탐샘플   #####
V-01→ V-04 24h소크 #####
A-01 이벤트연동 #####
A-02 환각로그   #####
A-03 골든추가   #####
A-05 강사검수   #####

마일스톤      M1→      M2→      M3
```

실제 시작주는 **현장 카메라·GPU·데이터 확보** 시점에 맞춰 당겨집니다.

상위 마일스톤 3개

마일스톤	목표	완료 판정(숫자)	선행 조건	예상 소요(추정)
M1. 계속 기반 확보	"지금 성능이 얼마인지"를 숫자로 확정	V-02 클래스별 F1 산출 + A-01 연동 설계 승인	weights 접근(V-01)	1~2주
M2. 현장 최소검증	실영상 오탐율 + 24h 안정성 실측	V-05 오탐표 + V-04 소크 합격 + A-05 강사검수	현장 카메라·GPU (외부)	현장 확보 후 2~3주
M3. 파일럿 준비완료	시험 설치 가능 상태	NO-Hardhat 개선 착수(V-03) + A-01 연동 실동작 + 폐쇄망 대안(A-07)	M1·M2	M2 후 (데이터 리드타임 의존)

교차 의존 지점 (한쪽이 늦으면 양쪽이 멈춤)

- ★ V-01(가중치 fail-fast) ↔ A-01(이벤트 자동반영): 검출이 풀백 없이 제대로 돌아야(V-01) 자동반영(A-01)이 신뢰를 가짐. **V-01이 늦으면 A-01도 무의미.**
- ★ V-05(현장 오탐) ↔ A-05(강사 검수): 실제 감지 결과라야 서류 품질이 현장 의미를 가짐. **현장 영상이 없으면 양쪽 다 멈춤.**
- GPU 장비 ↔ V-04·V-06·V-03: CUDA 없으면 24h 소크 신뢰·재측정·재학습이 전부 막힘.

최단 경로상의 병목 티켓 3개

- [V-01] 가중치 공유·fail-fast — 팀 확장·모든 측정의 전제(F-8).
- [V-02] 클래스별 F1 산출 — 이게 없으면 KISA 진척·개선 우선순위를 못 정함.
- [A-01] 이벤트→체크리스트 자동반영 — "AI 자동화" 핵심 가치, 파일럿 설득력의 심장.

되돌릴 수 없는 이벤트 — 순서를 못박음

아래는 한 번 하면 되돌릴 수 없으니 반드시 이 순서로 진행합니다.

- 특허 출원 (RIG 등 핵심 메커니즘)
↓ (출원 전 공개 시 특허 상실)
- 공개 시연 / 외부 발표
↓
- 파일럿 설치 (현장 개인정보 동의·노사협의 선행)
↓
- KISA 인증 신청 (F1 근거 확보 후)

- 특허 출원 → 공개 시연:** 출원 전 외부 공개는 특허를 잃습니다. 시연·발표·제안서 외부 배포 전 **출원 먼저**.
- 개인정보 동의·노사협의 → 파일럿 설치:** 근로자 영상은 동의·협의 없이 다루면 안 됩니다(docs/PRIVACY_POLICY_DRAFT.md).
- F1 근거 → KISA 신청:** 측정 없이 신청하면 탈락. V-02·V-03 완료가 선행.

40. Claude Code 사용법 ★ 바이브코딩 규칙

우리 팀은 Claude Code 하나만 씁니다. 이유: 도구가 여러 개면 팀원마다 환경·습관이 갈려 협업이 깨집니다. 하나로 통일하면 지시문·문제해결 방식을 공유할 수 있습니다.

설치·실행 (화면 흐름)

- 터미널을 엽니다 (macOS: Spotlight에서 "터미널" / Windows: PowerShell). 상세는 `50_완전초보_온보딩.md`.
- 프로젝트 폴더로 이동합니다.

```
cd ~/Desktop/VIGENT
```

실행하면 이런 게 보입니다:

(아무 메시지 없이 프롬프트가 `VIGENT` 폴더로 바뀝니다)

- Claude Code를 실행합니다.

```
claude
```

실행하면 이런 게 보입니다:

```
Claude Code
> (여기에 지시를 입력) |
```

좋은 지시문 4원칙

- 목적 먼저 — "무엇을 왜" 한 줄로.
- 컨트롤 파일 범위 명시 — 어느 파일까지 만져도 되는지.
- 완료 조건 제시 — 무엇이 되면 끝인지(숫자로).
- 한 번에 하나 — 여러 개를 한꺼번에 시키지 않기.

나쁜 지시문 vs 좋은 지시문 (이 프로젝트 실제 작업으로)

예시 1 — 정확도 측정

- 나쁨: "정확도 좀 바꿔"
- 좋음: "person·PPE·fire/smoke 클래스별 precision-recall·F1을 산출하는 스크립트를 `tools/` 에 만들어줘. 기존 `benchmarks/` 코드를 재활용하고, 측정 환경 (MPS/CPU)을 리포트에 병기해줘. 코드 수정 전 계획 먼저 보여줘." (V-02 티켓)

예시 2 — 추적기 교체

- 나쁨: "추적 더 좋게 바꿔"
- 좋음: "추적기 A/B 결과 SORT가 단편화가 심해. `rfdetr_service.py`·`rfdetr_zone_track.py`의 SORTTracker를 ByteTrackTracker로 교체하는 계획을 먼저 보여주고, 게이트(테스트 82) 통과 확인 후 커밋해줘. 다른 파일은 건드리지 마."

예시 3 — 연동 배선

- 나쁨: "감지된 거 서류에 자동으로 넣어줘"
- 좋음: "`data_engine.log_event` → `scribe.build_checklist` 경로를 조사해줘. 지금은 빈도만 재사용하는데, 감지 규칙이 체크리스트 항목을 자동 X로 반영하도록 바꾸는 설계를 **계획으로** 먼저 보여줘. 미감지 항목은 '수동확인' 유지. `golden_regression` 회귀 없게." (A-01 티켓)

작업 흐름 (반드시 이 순서)

① 지시 입력 → ② "계획 먼저 보여줘" 요청 → ③ 계획 확인 후 "진행해" 승인
→ ④ 결과·게이트 확인 → ⑤ 커밋

- 핵심: 코드를 바꾸는 지시는 **항상 계획을 먼저 받고** 승인하세요. 바로 실행시키지 않습니다.

CLAUDE.md 가 뭔가

- 프로젝트 루트의 `CLAUDE.md` 는 Claude Code가 매 대화마다 자동으로 읽는 규칙서입니다.
- 여기 규칙을 적어두면 모든 팀원의 Claude Code가 같은 규칙으로 동작합니다.

- 규칙 추가법: "CLAUDE.md에 '~~ 규칙을 추가해줘'"라고 지시하면 됩니다. (단, 팀 합의 후)

세션 관리

- 대화가 길어지면 **한 작업 끝 → 커밋 → 새 대화 시작**. 짧게 끊어야 Claude가 헛갈리지 않습니다.
- **작업 단위로 커밋**하세요. 한 커밋에 여러 작업을 몰지 마세요.

절대 하지 말 것

- ❌ 확인 없이 **대량 파일 수정**을 승인하기
- ❌ 이해 못 한 코드를 커밋하기 (뭐 하는지 물어보고 이해한 뒤 커밋)
- ❌ 에러를 숨기고 다음 단계로 넘어가기 (에러는 반드시 해결·공유)

막혔을 때 3단계

1. **에러 메시지 전문**을 복사 (일부만 말고 전체).
2. **Claude Code에 붙여넣기** + "이 에러가 났어. 원인과 해결을 알려줘".
3. 그래도 안 되면 **팀 채널에 에러 전문 공유**.

50. 완전초보 온보딩 (가장 자세하게)

컴퓨터에 아무것도 안 깔린 상태를 가정합니다. macOS와 Windows를 나눠서 씁니다. 상세 개발 환경은 docs/ONBOARDING.md 도 함께 참고합니다(이 문서는 그 초보자용 버전입니다).

❌ 이거 없으면 아무것도 안 됩니다 (제일 먼저)

git으로 코드를 받아도 아래 2개는 안 따라옵니다. 반드시 팀에서 따로 받아 넣으세요.

① .env (비밀키 파일)

- git에 올리면 안 되는 API 키가 들어갑니다 → **각자 만들어야** 합니다.

```
cp .env.example .env
```

실행하면 이런 게 보입니다:

(아무 메시지 없이 .env 파일이 생깁니다)

- 그다음 .env 를 열어 팀에게 받은 키를 채웁니다(예: OPENAI_API_KEY). 키는 절대 채팅·git에 쓰지 마세요.

② 모델 가중치 weights/ (812MB, 18개 파일)

- 용량이 커서 git에 없습니다. 없으면 검출이 엉뚱하게 동작(COCO 풀백)해 정확도가 급락합니다(과거 F-8 사고).
- 팀 공유 위치(클라우드 드라이브/USB)에서 받아 **vigent-core/weights/** 폴더에 넣습니다.
- 확인:

```
ls vigent-core/weights/ | wc -l
```

실행하면 이런 게 보입니다:

18

[스크린샷 자리] weights 폴더에 파일 18개가 들어있는 모습

이 2개가 준비되기 전에는 서버가 제대로 안 돌립니다. 아래 설치를 먼저 하고, 이 2개로 마무리하세요.

macOS

1. 터미널 열기

- Cmd(⌘) + Space → "터미널" 입력 → Enter.

[스크린샷 자리]

2. 내 위치 확인·이동·목록

```
pwd          # 지금 어느 폴더인지
cd ~/Desktop/VIGENT  # VIGENT 폴더로 이동
ls           # 폴더 안 목록 보기
```

실행하면 이런 게 보입니다:

```
/Users/본인이름/Desktop/VIGENT
CLAUDE.md  README.md  vigent-core  docs  ...
```

3. Git 확인·설치·설정

```
git --version          # 설치돼 있으면 버전이 뜸
```

- 안 깔렸으면: `xcode-select --install` 실행 → 설치 창을 따라갑니다.
- 최초 1회 설정:

```
git config --global user.name "본인이름"
git config --global user.email "본인이메일"
```

4. Python·가상환경·의존성

```
python3 --version      # 3.11~3.13 권장
python3 -m venv .venv   # 가상환경 만들기
source .venv/bin/activate # 활성화 (앞에 (.venv) 가 붙음)
pip install -r requirements.txt # 의존성 설치
```

- opencv 정리(중요, docs/ONBOARDING.md §2.2와 동일):

```
pip uninstall -y opencv-python opencv-contrib-python
pip install --force-reinstall --no-deps opencv-contrib-python-headless==4.13.0.92
```

5. 리포 내려받기 (처음 한 번만)

```
git clone <여기에 저장소 주소> VIGENT
cd VIGENT
```

6. Claude Code 설치·실행

```
claude          # 프로젝트 폴더에서 실행
```

[스크린샷 자리] Claude Code 시작 화면

7. 첫 실행 성공 확인

- 서버 켜기:

```
./run.sh
```

- 다른 터미널에서:

```
curl http://127.0.0.1:8010/health
```

실행하면 이런 게 보이면 성공입니다:

```
{"status":"ok","loaded":true, ... }
```

- `"loaded":true` 가 보이면 모델이 제대로 올라간 것입니다. (안 뜨면 §가중치 확인)

Windows

⚠ 이 프로젝트는 macOS에서 개발됐습니다. Windows에서 처음 켤 때 잔가지 조정이 필요할 수 있습니다. NVIDIA GPU가 있으면 검출이 더 빠르고 재학습도 가능합니다 (상세 docs/INFRA_REQUIREMENTS.md).

1. 터미널(PowerShell) 열기

- 시작 버튼 → "PowerShell" 입력 → Enter.

[스크린샷 자리]

2. 내 위치 확인·이동·목록

VIGENT 대외비 · 초안—검토용 v0.9

```
Get-Location          # 지금 폴더 (pwd 대신)
cd $HOME\Desktop\VIGENT # 이동
dir                   # 목록 (ls 대신 dir)
```

3. Git 확인·설치·설정

```
git --version
```

- 안 깔렸으면: <https://git-scm.com> 에서 설치.

```
git config --global user.name "본인이름"
git config --global user.email "본인이메일"
```

4. Python·가상환경·의존성

```
python --version          # 3.11~3.13
python -m venv .venv
.venv\Scripts\activate    # 활성화 (macOS와 경로 다름)
pip install -r requirements.txt
```

- ⚠ 로컬 VLM(mlx)은 애플 전용이라 Windows에서 설치·실행 안 됩니다. `requirements-optional.txt` 의 `mlx` 줄은 건너뛰고, VLM은 클라우드(`OPENAI_API_KEY` + `.env` 의 `VIGENT_CLOUD_VLM=1`)로 씁니다.
- ⚠ NVIDIA GPU면 torch를 CUDA용으로 따로 설치하세요(자세히는 `docs/INFRA_REQUIREMENTS.md`).

5·6·7. clone·Claude·실행

```
git clone <여기에 저장소 주소> VIGENT
cd VIGENT
claude
```

- 서버 실행 후 확인:

```
curl http://127.0.0.1:8010/health
```

"loaded":true 가 보이면 성공.

자주 나는 에러 5가지와 대처

증상	원인	대처
Permission denied	실행 권한 없음	<code>chmod +x run.sh</code> (mac) / 관리자 PowerShell
command not found: claude / python	미설치·경로 문제	재설치, 터미널 새로 열기
Address already in use (포트 8010)	이미 서버가 떠 있음	기존 프로세스 종료 후 재실행
ModuleNotFoundError	가상환경 미활성	<code>source .venv/bin/activate</code> (mac) / <code>.venv\Scripts\activate</code> (win)
검출이 안 됨 / "loaded":false	모델 가중치 없음	<code>vigent-core/weights/</code> 에 812MB 복사(맨 위 ② 참고)

막혔을 때 행동 순서

1. 에러 메시지 전체를 복사한다 (일부만 X).
2. Claude Code에 "이 에러가 났어" + 붙여넣기 → 원인·해결 요청.
3. 그래도 안 되면 팀 채널에 에러 전문을 공유한다.

60. 협업 규칙 — Git

깃(git)이 뭔가 (비유)

- 게임 세이브 파일과 같습니다. 작업을 "저장(커밋)"하면 언제든 그 지점으로 돌아갈 수 있습니다.
- 여러 명이 각자 세이브를 만들고 나중에 합치는(merge) 도구입니다. 문서 버전관리(v1, v2 ...)를 자동으로 해준다고 보면 됩니다.

반드시 지킬 5가지 규칙 (제일 중요)

- 작업 전 `git pull` — 남이 올린 최신을 먼저 받는다.
- 내 작업은 내 브랜치에서 — `main` 에 직접 작업하지 않는다.
- 작업 단위로 커밋 — 한 번에 하나. 메시지에 무엇을 했는지 쓴다.
- `.env` ·가중치·현장영상은 절대 올리지 않는다.
- 막히면 멈추고 공유 — 특히 충돌(conflict)은 혼자 해결하려 하지 말 것.

하루 작업 흐름 (명령어 + 한 줄 해설)

```
git pull                                # 최신 내용 받기
git checkout -b feat/내작업이름        # 내 작업 공간(브랜치) 만들기
# ... 작업 ...
git status                              # 뭐가 바뀌었는지 보기
git add -A                             # 바뀐 것 전부 담기
git commit -m "feat: 무엇을 했는지"    # 저장(커밋) 하기
git push -u origin feat/내작업이름     # 내 브랜치를 서버에 올리기
```

브랜치 이름 규칙

- 형식: 종류/짧은설명 — 예: `feat/f1-측정`, `fix/포트충돌`, `docs/온보딩보완`

커밋 메시지 규칙 (종류 + 한글 설명)

- `feat`: 새 기능 / `fix`: 버그 수정 / `docs`: 문서 / `test`: 테스트 / `refactor`: 구조 개선
- 예시 5개:
 - `feat`: 클래스별 F1 산출 스크립트 추가 (V-02)
 - `fix`: 가중치 부재 시 fail-fast 전환 (F-8)
 - `docs`: 신규 팀원 온보딩 문서 추가
 - `test`: rig_monitor 상태전이 테스트 보강
 - `refactor`: 이벤트→체크리스트 연동 구조 정리 (A-01)

절대 치면 안 되는 명령어와 이유

명령어	왜 위험한가
<code>git push --force</code>	남의 작업을 덮어써 삭제할 수 있음
<code>git reset --hard</code>	저장 안 한 내 작업이 되돌릴 수 없이 사라짐
<code>rm -rf</code>	폴더·파일을 복구 불가능하게 삭제
<code>main</code> 에 직접 push	리뷰 없이 전원에게 영향, 사고 시 추적 어려움

충돌(conflict)이 났을 때 (초보자)

- 당황하지 마세요. 흔한 일입니다.
- 아무것도 저장하지 말고 상황 그대로 둡니다.
- 화면(충돌 메시지)을 Claude Code에 그대로 붙여넣고 "충돌이 났어. 어떻게 해결하는지 알려줘"라고 물어봅니다.

git에 올리면 안 되는 것 (다시)

- `.env`, API 키·비밀번호
- 모델 가중치(weights/, 812MB)
- 현장 영상·개인정보 포함 파일
- (이미 `.gitignore` 가 대부분 막아두지만, 새 파일 추가 시 주의)

코드 리뷰 (PR)

VIGENT 대외비 · 초안—검토용 v0.9

- 작업을 올린 뒤 GitHub에서 PR(Pull Request, 병합 요청)을 만듭니다.

- 리뷰어를 지정합니다(내 풀더 담당 + 공유부면 양쪽 — 70_역할분담 참고).
- 승인 없이 merge 금지. 최소 1명 승인 후 합칩니다.

70. 역할 분담과 주간 루틴

역할과 담당 폴더 (파일 충돌을 구조적으로 예방)

감사 결과 비전·에이전트는 서로 import하지 않아(느슨한 결합) 폴더로 나누면 충돌이 거의 없습니다.

역할	담당 폴더/파일	대표 티켓
비전(L1)	vigent-core/rfdetr_service.py · worker.py · proximity.py · ergonomics.py · zone_*.py · routers/{detect,zone,ppe}.py · weights/	V-01~V-08
에이전트(L2)	vigent-core/agents/ · routers/{safety_core,tbm,incident}.py · eval/golden/ · config/corpus/	A-01~A-08
문서·사업	docs/ · docs/team/ · 제안서·조달	제안·인프라·개인정보
현장검증	현장 영상 수집·오탐 분류·라벨(데이터, 코드 아님)	V-05-A-05
공유부(양쪽 리뷰 필수)	app_state.py · web_util.py · main.py · data_engine.py · vision.yaml	—

- 공유부를 바꿀 땐 반드시 상대 역할과 조율하고 PR 리뷰를 양쪽에서 받습니다.
- (참고) CODEOWNERS 파일로 이 규칙을 자동화할 수 있습니다 — GitHub에 올린 뒤 도입 검토.

주 1회 30분 동기화 미팅 — 아젠다 템플릿

- 지난주 완료 티켓 (각자 1분) — 5분
- 병목 3개(V-01·V-02·A-01) 진행 점검 — 10분
- 교차 의존 막힘 확인 (한쪽 늦으면 양쪽 정지) — 5분
- 이번 주 각자 목표 티켓 1개 선언 — 5분
- 위험 신호(조기 경보) 공유 — 5분

데일리 3줄 보고 양식 (팀 채널)

[이름] YYYY-MM-DD
어제 한 것 : (티켓 번호 + 한 줄)
오늘 할 것 : (티켓 번호 + 한 줄)
막힌 것 : (없으면 '없음')

티켓 진행 상태 관리 — 방식 선택

- A. 문서 체크박스: 이 문서들의 ☐ 체크박스를 직접 채움. 간단하나 여러 명이 동시에 고치면 충돌.
- B. GitHub Issues: 티켓 하나 = 이슈 하나. 담당·상태·댓글이 자동 기록되고 PR과 연결됨.

권장: B(GitHub Issues). 이유 — (1) 충돌 없음(각 이슈 독립), (2) 누가 뭘 하는지 한눈에, (3) PR·커밋과 자동 연결로 이력이 남음. 초보 팀도 "이슈 만들고 → 담당 정하고 → 완료 시 닫기"만 익히면 됩니다.

팀원별 "첫 주에 이것만 하면 성공"

역할	첫 주 목표
비전(L1)	V-01 완료(가중치 공유 경로 확정 + fail-fast) — 팀 확장의 1번 장애물 제거
에이전트(L2)	A-04(Coach 범위 확정) or A-03(골든 1건 추가) 중 1개
문서·사업	제안서(docs/PILOT_PROPOSAL.md) 현장 맞춤 1버전 다듬기
현장검증	현장 영상 확보 경로 1개 확정(카메라·연락처)
전원	50_완전초보_온보딩 따라 서버 /health 에 loaded:true 띄우고 첫 커밋

80. 용어 사전

이 리포에서 실제로 쓰이는 용어만 모았습니다. 각 항목 = 한 줄 정의 + "우리 프로젝트에선 이렇게".

개발·협업

용어	정의	우리 프로젝트에선
커밋(commit)	작업 내용을 한 시점으로 저장하는 것	"작업 단위로 커밋" — 한 번에 하나
브랜치(branch)	내 작업만 담는 분리된 공간	<code>feat/내작업이름</code> 으로 만들어 작업
PR(Pull Request)	내 브랜치를 main에 합쳐달라는 요청	승인 없이 merge 금지
머지(merge)	브랜치를 합치는 것	PR 승인 후에만
게이트(gate)	커밋 전 통과해야 하는 검사	ruff·mypy·테스트 82·OpenAPI 무변경
스텝(step)	검대기만 있고 실행 안 하는 코드	Coach 에이전트(12줄)
fail-fast	문제 시 조용히 넘어가지 않고 즉시 멈춤	가중치 없으면 폴백 대신 멈추기(V-01)
폴백(fallback)	실패 시 대체 동작	가중치 없으면 COCO로 폴백(주의: 정확도↓, F-8)

비전(L1)

용어	정의	우리 프로젝트에선
mAP	검출 정확도 종합 지표(0~1)	fire/smoke 공개셋 mAP@50 80.13%
precision(정밀도)	잡았다고 한 것 중 맞은 비율(오탐의 반대)	대부분 미측정 — 재측정 필요
recall(재현율)	실제 위험 중 잡아낸 비율(미탐의 반대)	PPE·person 수치는 대부분 recall만
F1	precision과 recall의 조화평균	KISA 기준 F1≥0.90 (현재 미산출, V-02)
추론 지연(latency)	한 프레임 처리에 걸리는 시간(ms)	타일 +415ms/f(CPU)
RF-DETR	우리가 쓰는 객체 검출 모델	검출 코어(<code>rfdetr_service.py</code>)
포즈 추정(pose)	사람 관절 위치 추정	RTMPose/yolov8n-pose (<code>ergonomics.py</code>)
추적(tracking)	프레임 간 같은 사람에 같은 ID 부여	ByteTrack(SORT에서 교체)
ByteTrack	우리가 채택한 추적 알고리즘	ID 스위치 0으로 개선
타일링	위험구역만 잘라 재검출해 소형작업자를 잡는 기법	recall 0→89~95%(단 +415ms)
MPS	애플 칩(맥)의 GPU 가속	현재 개발환경(배포는 CUDA)
CUDA	NVIDIA GPU 가속	배포·재학습 목표 환경
엣지(edge)/RK3588	현장 저전력 소형 처리장치	포팅 검토 단계(코드 없음)

에이전트(L2)

용어	정의	우리 프로젝트에선
에이전트(agent)	특정 역할을 맡는 AI 모듈	Scribe·Copilot·Guard 등 6개
골든셋(golden set)	정답이 있는 평가용 표본	체크리스트 3건 + 홀드아웃 15케이스
재정 엔진	산출물을 정답과 비교해 점수화	<code>golden_regression.py</code>
베이스라인	회귀(나빠짐) 감지 기준선	품질 합격선 아님 — 정답 확정 아님
화이트리스트	허용된 법령 조문 목록	밖의 조문 인용은 차단(F-11)
환각(hallucination)	AI가 없는 사실·조문을 지어냄	crane 골든서 4건 관찰
상태기계	조건에 따라 상태가 전이되는 로직	RIG 모듈(세부는 대외비)
LLM	대형 언어모델(텍스트 생성 AI)	OpenAI/Anthropic(외부 API)
온프레임(on-prem)	외부 클라우드 없이 자체 장비에서 실행	폐쇄망 대응 과제(현재 미지원)
폐쇄망	인터넷이 차단된 내부망	공장 다수의 조건(S5)

사업·인증

용어	정의	우리 프로젝트에선
파일럿(pilot)	유상 전 시범 설치·검증	위험구역 침입 단일기능부터
KISA 인증	지능형 CCTV 성능 인증(F1≥0.90)	S4 목표
TTA GS	SW 품질 인증 등급	S5(조달 진입)
조달 등록	공공조달 판매 자격	S5